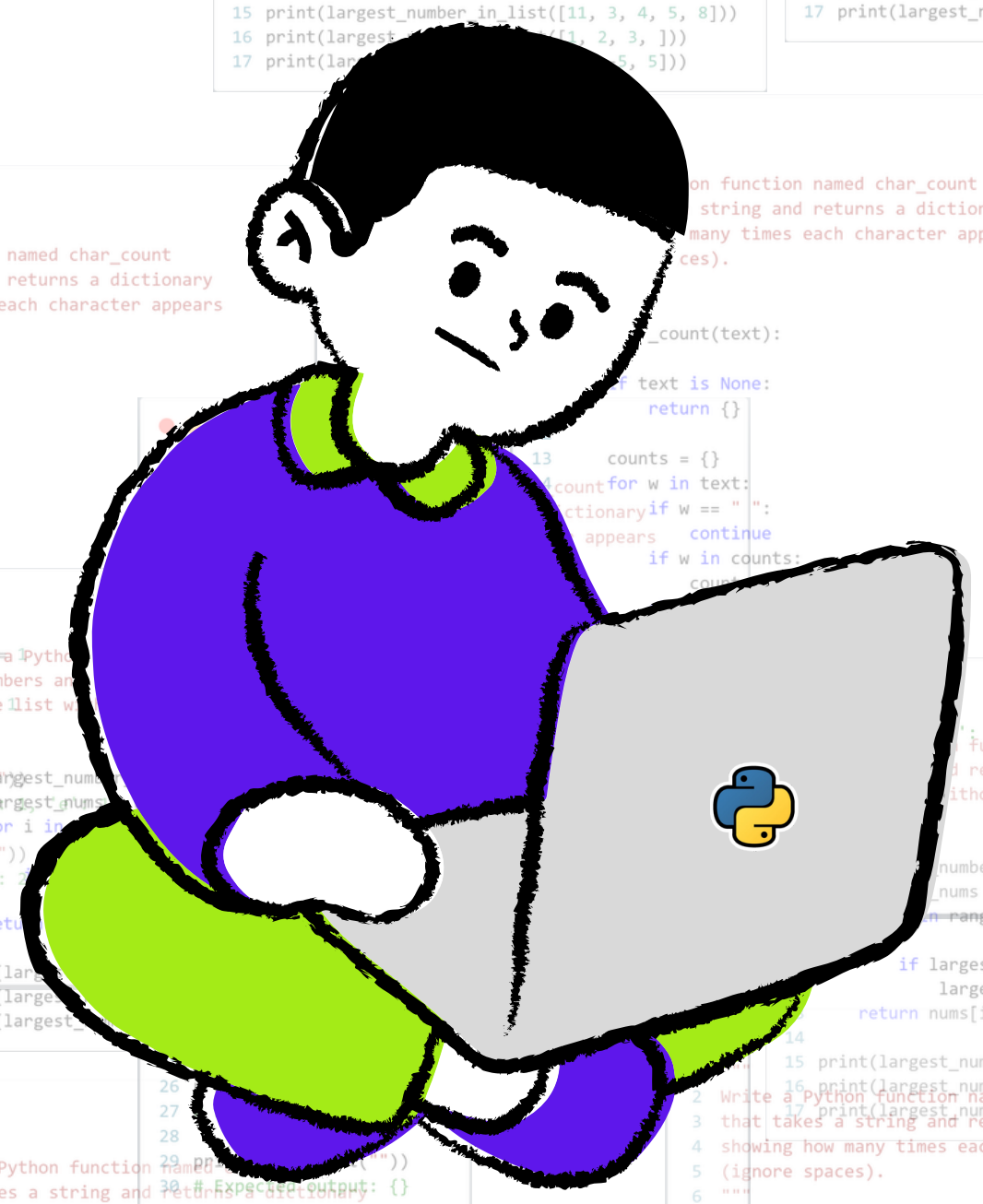
Feb 26

PART 1



PYTHON PRACTICE QUESTIONS

Rudra Prasad Bhuyan

```
"""
Write a Python function that takes a list
of numbers and returns the largest number
in the list without using max().
"""

def largest_number_in_list(nums):
    largest_nums = nums[0]
    for i in range(len(nums)):

        if largest_nums > nums[i]:
            largest_nums = nums[i]
    return nums[i]

print(largest_number_in_list([11, 3, 4, 5, 8]))
print(largest_number_in_list([1, 2, 3, ]))
print(largest_number_in_list([10, -5, 5]))
```

```
"""
Write a Python function that takes a list
of numbers and returns the largest number
in the list without using max().
"""

def largest_number_in_list(nums):
    largest_nums = nums[0]
    for n in nums:
        if n > largest_nums:
            largest_nums = n
    return largest_nums


print(largest_number_in_list([11, 3, 4, 5, 8]))
print(largest_number_in_list([1, 2, 3, ]))
print(largest_number_in_list([10, -5, 5]))
```

```
"""
Write a Python function named count_even_numbers
that takes a list of integers as input and returns
the count of even numbers in the list.
Do not use count() or any built-in shortcuts.
"""

def count_even_numbers(nums):
    total = 0
    for n in nums:
        if n % 2 == 0:
            total += 1
    return total

print(count_even_numbers([1, 2, 3, 4, 5, 6]))#  3
print(count_even_numbers([10, 20, 30])) # 3
print(count_even_numbers([1, 3, 5])) # 0

```

```python
"""
Write a Python function named count_vowels
that takes a string and returns the number
of vowels (a, e, i, o, u) in it.
Uppercase and lowercase both count.
"""

# M1
def count_vowels(text):
    count = 0
    for t in text.lower():
        if t in ['a', 'e', 'i', 'o', 'u']:
            count+=1

    return count

# M2
def count_vowels(text):
    vowels = {'a', 'e', 'i', 'o', 'u'}
    count = 0
    for ch in text.lower():
        if ch in vowels:
            count += 1
    return count

print(count_vowels("hello")) # 2
print(count_vowels("PYTHON")) # 1
print(count_vowels("bcdfg")) # 0
```

```python
"""
Write a Python function named remove_duplicates
that takes a list and returns a new list with
duplicates removed while keeping the original order.
"""

# M1
def remove_duplicates(nums):
    unique_list = []
    for n in nums:
        if n not in unique_list:
            unique_list.append(n)
    return unique_list

# M2
def remove_duplicates(nums):
    seen = set()
    results = []
    for n in nums:
        if n not in seen:
            seen.add(n)
            results.append(n)
    return results


print(remove_duplicates([1, 2, 2, 3, 1, 4]))
# Expected output: [1, 2, 3, 4]

print(remove_duplicates([5, 5, 5]))
# Expected output: [5]

print(remove_duplicates([]))
# Expected output: []

```

```python
"""
Write a Python function named word_count
that takes a string and returns a dictionary
where the keys are words and the values are
how many times each word appears.
"""

def word_count(text):
    if not text:
        return {}

    words = text.lower().split(" ")
    counts = {}

    for w in words:
        if w in counts:
            counts[w] += 1
        else:
            counts[w] = 1
    return counts


print(word_count("apple banana apple"))
# Expected output: {'apple': 2, 'banana': 1}

print(word_count("Python python PYTHON"))
# Expected output: {'python': 3}

print(word_count(""))
# Expected output: {}

```

```python
"""
Write a Python function named char_count
that takes a string and returns a dictionary
showing how many times each character appears
(ignore spaces).
"""

def char_count(text):

    if text is None:
        return {}

    counts = {}
    for w in text:
        if w == " ":
            continue
        if w in counts:
            counts[w] += 1
        else:
            counts[w] = 1
    return counts

print(char_count("hello"))
# Expected output: {'h': 1, 'e': 1, 'l': 2, 'o': 1}

print(char_count("a a b"))
# Expected output: {'a': 2, 'b': 1}

print(char_count(""))
# Expected output: {}
```

```
"""
Write a function named count_positive_negative
that takes a list of numbers and returns a
dictionary with counts of positive, negative,
and zero numbers.
"""

def count_positive_negative(nums):
    count = {"positive": 0, "negative": 0, "zero": 0}

    if nums is None:
        return count

    for n in nums:
        if n > 0:
            count["positive"] += 1
        elif n < 0:
            count["negative"] += 1
        else:
            count["zero"] +=1

    return count


print(count_positive_negative([1, -2, 0, 3, -1, 0]))
# Expected output: {'positive': 2, 'negative': 2, 'zero': 2}

print(count_positive_negative([]))
# Expected output: {'positive': 0, 'negative': 0, 'zero': 0}

```

```
"""
Write a function named group_by_length
that takes a list of words and returns
a dictionary where:

- the key is the word length
- the value is a list of words of that length
"""

def group_by_length(words):
    if not words:
        return {}

    count = {}

    for w in words:
        l = len(w)
        if l in count:
            count[l].append(w)
        else:
            count[l] = [w]

    return count

print(group_by_length(["hi", "hy", "hello", "hey", "a"]))
# Expected output: {2: ["hi", "hy"], 5: ['hello'], 3: ['hey'], 1: ['a']}

print(group_by_length([]))
# Expected output: {}

print(group_by_length(None))
# Expected output: {}

```

```
"""
Write a function named group_by_first_letter
that takes a list of words and groups them by
their first character.
"""

def group_by_first_letter(words:list) -> dict:
    if not words:
        return {}

    count_dict = {}

    for w in words:
        key = w[0]
        if key in count_dict:
            count_dict[key].append(w)
        else:
            count_dict[key] = [w]

    return count_dict

print(group_by_first_letter(["apple", "ant", "banana", "bat"]))
# Expected output: {'a': ['apple', 'ant'], 'b': ['banana', 'bat']}

print(group_by_first_letter([]))
# Expected output: {}

print(group_by_first_letter(None))
# Expected output: {}

```

```
"""
Write a function named group_even_odd that
takes a list of integers and returns:
{
  "even": [...],
  "odd": [...]
}

"""
def group_even_odd(nums):
    even_odd_dict = {"even": [], "odd": []}

    if not nums:
        return even_odd_dict

    for n in nums:
        if n %2 == 0:
            even_odd_dict['even'].append(n)
        else:
            even_odd_dict['odd'].append(n)

    return even_odd_dict

print(group_even_odd([1, 2, 3, 4, 5, 6]))
# {'even': [2, 4, 6], 'odd': [1, 3, 5]}

print(group_even_odd([]))
# {'even': [], 'odd': []}

```

```
"""
Write a function named group_words_by_vowel_start
that takes a list of words and groups them
into two lists:

{
  "vowel": [...],
  "consonant": [...]
}

"""

def group_words_by_vowel_start(words):
    result = {"vowel": [], "consonant": []}
    vowels = {'a', 'e', 'i', 'o', 'u'}

    if not words:
        return result

    for w in words:
        first_char  = w[0].lower()
        if first_char  in vowels:
            result['vowel'].append(w)
        else:
            result['consonant'].append(w)

    return result

print(group_words_by_vowel_start(["apple", "banana", "orange", "grape", "umbrella"]))
# {'vowel': ['apple', 'orange', 'umbrella'], 'consonant': ['banana', 'grape']}

print(group_words_by_vowel_start([]))
# {'vowel': [], 'consonant': []}

print(group_words_by_vowel_start(None))
# {'vowel': [], 'consonant': []}

```

```
"""
Write a function named group_numbers_by_digits
that takes a list of integers and groups them
by number of digits.
"""


def group_numbers_by_digits(nums):
    if not nums:
        return {}

    result = {}
    for n in nums:
        count = len(str(n))
        if count in result:
            result[count].append(n)
        else:
            result[count] = [n]
    return result


print(group_numbers_by_digits([1, 22, 333, 4, 55, 6666]))
# {1: [1, 4], 2: [22, 55], 3: [333], 4: [6666]}

print(group_numbers_by_digits([]))
# {}

print(group_numbers_by_digits(None))
# {}
```

```
"""
Write a function named is_sorted_ascending
that takes a list of numbers and returns
True if the list is sorted in ascending order,
otherwise False.
"""

def is_sorted_ascending(nums):
    if not nums:
        return True

    sort_nums = sorted(nums)
    return nums == sort_nums

print(is_sorted_ascending([1, 2, 3, 4])) # True
print(is_sorted_ascending([1, 3, 2])) # False
print(is_sorted_ascending([])) # True
print(is_sorted_ascending([5])) # True

```

```python
"""
Write a function named square_even_numbers
that takes a list of numbers and returns a
new list containing the square of only the
even numbers.
"""

def square_even_numbers(nums):
    if not nums:
        return []

    even_list = []

    for n in nums:
        if n % 2 == 0:
            nn = n** 2
            even_list.append(nn)

    return even_list


print(square_even_numbers([1, 2, 3, 4, 5]))
# [4, 16]

print(square_even_numbers([]))
# []

print(square_even_numbers([1, 3, 5]))
# []
```

```python
"""
Write a function named has_duplicates that
returns True if a list contains any duplicate
values, otherwise False.
"""
# M1
def has_duplicates(nums):
    if not nums:
        return False
    unique = []

    for w in nums:
        if w not in unique:
            unique.append(w)

    return nums != unique

# M2
def has_duplicates(nums):
    seen = set()
    for n in nums:
        if n in seen:
            return True
        seen.add(n)
    return False

print(has_duplicates([1, 2, 3, 4])) # False
print(has_duplicates([1, 2, 3, 2])) # True
print(has_duplicates([])) # False
print(has_duplicates([5])) # False

```

```python
"""
Write a function named reverse_list
that reverses a list without using
.reverse().
"""

def reverse_list(nums):
    return nums[::-1]

print(reverse_list([1, 2, 3, 4])) # [4, 3, 2, 1]
print(reverse_list([])) # []
print(reverse_list([5])) # [5]

```

```
"""
Write a function named running_sum that
returns a list where each element is the
sum of all previous elements including itself.
"""

def running_sum(nums):
    new_list = []
    total = 0
    for n in nums:
        total += n
        new_list.append(total)
    return new_list

print(running_sum([1, 2, 3, 4])) # [1, 3, 6, 10]
print(running_sum([])) # []
print(running_sum([5])) # [5]

```

```
"""
Write a function named second_largest that returns
the second largest number in a list.

- Do not use sorted()
- Assume the list has at least two distinct numbers
"""

def second_largest(nums):
    first = float('-inf')
    second = float('-inf')

    for n in nums:
        if n > first:
            second = first
            first = n
        elif n < first and n > second:
            second = n

    return first, second

print(second_largest([10, 5, 20, 8])) # 10
print(second_largest([1, 2, 3, 4])) # 3
print(second_largest([5, 5, 3, 2, 5])) # 3
```

```
"""
Write a function named max_difference that returns
the maximum difference between two numbers in a list
such that the larger number comes after the smaller one
without using sort.
Ex-
max_difference([2, 3, 10, 6, 4, 8, 1]) # 8  (10 - 2)
"""

def max_difference(nums):
    min_so_far = nums[0]
    max_diff = nums[1] - nums[0]

    for i in range(1, len(nums)):
        max_diff = max(max_diff, nums[i] - min_so_far)
        min_so_far = min(nums[i], min_so_far)

    return max_diff

print(max_difference([2, 3, 10, 6, 4, 8, 1]))

```

Thank you

Stay connected with data ...

Rudra Prasad Bhuyan